Steam Player Behavior Segmentation

Using Unsupervised Machine Learning

Levente Szabó
Data Programming

October 2026

1 Problem Definition

1.1 Background and Motivation

Steam is a digital gaming platform where users can own multiple games and accumulate different amounts of recorded playtime.

However, users may exhibit substantially different gaming behaviors. Some users own large game libraries but have relatively little recorded playtime, while others spend many hours playing a smaller number of games.

Understanding these behavioral differences may help identify meaningful patterns in player engagement.

This project investigates whether Steam users can be divided into distinct groups based on their game ownership and recorded playtime behavior.

1.2 Research Question

The main research question of this project is:

> **Can Steam users be grouped into distinct segments based on their game ownership and recorded playtime behavior?**

The analysis aims to identify similarities and differences between users by examining features such as the number of games owned, the number of games played, and recorded playtime statistics.

Project Availability

The complete project, including the Python scripts, datasets, and analysis, is available on GitHub:

`https://github.com/NagyLevin/SteamProject`

2 Dataset Source and Description

2.1 Dataset Source and Overview

I selected the *Steam Video Games* dataset by the Tamber Team from Kaggle, originally collected for gaming behavior analysis and recommendation systems.

Source: `https://www.kaggle.com/datasets/tamber/steam-video-games`

The original `steam-200k.csv` contains 200,000 records, five columns, 12,393 unique users, and 5,155 distinct games.

Each row represents a user-game interaction: `purchase` indicates ownership, while `play` records playtime in hours.

The dataset contains 129,511 purchase records and 70,489 play records.

I selected this dataset because it provides both ownership and playtime information, allowing different player behaviors to be investigated.

2.2 Dataset Variables

The original CSV has no header row, so I assigned column names during data loading.

Table 1: Original Dataset Variables

Variable	Data Type	Description
`user_id`	Integer	Unique identifier of a Steam user.
`game_name`	String	Name of the video game associated with the interaction.
`behavior`	String	Type of interaction, either `purchase` or `play`.
`value`	Float	Equals 1.0 for purchase records; represents recorded playtime in hours for play records.
`extra`	Integer	Additional column containing only zero values in the original dataset.

The `value` column has different meanings depending on the interaction type, while `extra` contains only zeros.

Figure 1 shows a sample of the original interaction records.

	user_id	game_name	behavior	value	extra
0	151603712	The Elder Scrolls V Skyrim	purchase	1.0	0
1	151603712	The Elder Scrolls V Skyrim	play	273.0	0
2	151603712	Fallout 4	purchase	1.0	0
3	151603712	Fallout 4	play	87.0	0
4	151603712	Spore	purchase	1.0	0
5	151603712	Spore	play	14.9	0
6	151603712	Fallout New Vegas	purchase	1.0	0
7	151603712	Fallout New Vegas	play	12.1	0
8	151603712	Left 4 Dead 2	purchase	1.0	0
9	151603712	Left 4 Dead 2	play	8.9	0
10	151603712	HuniePop	purchase	1.0	0
11	151603712	HuniePop	play	8.5	0
12	151603712	Path of Exile	purchase	1.0	0
13	151603712	Path of Exile	play	8.1	0
14	151603712	Poly Bridge	purchase	1.0	0
15	151603712	Poly Bridge	play	7.5	0
16	151603712	Left 4 Dead	purchase	1.0	0
17	151603712	Left 4 Dead	play	3.3	0
18	151603712	Team Fortress 2	purchase	1.0	0
19	151603712	Team Fortress 2	play	2.8	0
20	151603712	Tomb Raider	purchase	1.0	0
21	151603712	Tomb Raider	play	2.5	0
22	151603712	The Banner Saga	purchase	1.0	0
23	151603712	The Banner Saga	play	2.0	0

Figure 1: Preview of the original Steam dataset.

3 Data Quality Check

I examined the original 200,000 Steam records for missing values, duplicates, invalid values, purchase-play consistency, and unnecessary columns using Python, Pandas, and NumPy.

3.1 Missing Value Assessment

I used `isna()` to count missing values and calculate their percentages.

```
# Inside get_missing_values()
missing_count = df.isna().sum()
missing_percent = df.isna().mean() * 100
```

No missing values were found in any of the five original columns.

3.2 Duplicate Record Assessment

I used `duplicated()` to identify identical records and examined their interaction types.

The main operations from `get_duplicates()` and `inspect_duplicate_records()` are:

```
# Inside get_duplicates()
duplicate_count = int(df.duplicated().sum())

# Inside inspect_duplicate_records()
duplicate_rows = df[df.duplicated(keep=False)]

duplicates_by_behavior = (
    df[df.duplicated()]["behavior"].value_counts()
)
```

The analysis identified 707 fully duplicated records (0.35%), all belonging to the `purchase` category.

3.3 Repeated Playtime Records

I also checked whether the same user-game pair had multiple playtime records.

```
play_records = df[df["behavior"] == "play"]

repeated_play_records = play_records[
    play_records.duplicated(
        subset=["user_id", "game_name"],
        keep=False
    )
]
```

I found 24 play records belonging to 12 repeated user-game pairs with different playtime values.

Unlike exact duplicates, these records required a separate cleaning decision.

3.4 Numerical Value Validation

I used `value_check()` to validate interaction types, numerical values, and recorded playtime.

Key operations are shown below.

```
# Inside value_check()
invalid_behavior = ~df["behavior"].isin(
    ["purchase", "play"]
)

invalid_numeric = ~np.isfinite(df["value"])

invalid_purchase = purchase_df["value"] != 1.0
negative_playtime = play_df["value"] < 0
zero_playtime = play_df["value"] == 0
```

No invalid behavior types, non-finite values, incorrect purchase values, or non-positive playtime records were identified.

Recorded playtime ranged from 0.1 to 11,754 hours. The maximum was unusually high but was not considered an error without further evidence.

3.5 Purchase-Play Relationships

I used Python sets to compare unique user-game combinations with purchase and play records.

The main operations from `get_purchase_play_relationships()` are shown below.

```
# Create sets using user_id and game_name
# ... purchase_pairs and play_pairs

both = purchase_pairs & play_pairs
purchase_only = purchase_pairs - play_pairs
play_only = play_pairs - purchase_pairs
```

The dataset contained 128,804 unique user-game pairs: 70,477 with both purchase and play records, and 58,327

with purchase records only.

No play record existed without a corresponding purchase record, supporting the later calculation of the play ratio.

3.6 Unnecessary Column Assessment

I used `find_extra_column()` to investigate the final column.

```
# Inside find_extra_column()
extra_col = df["extra"]

unique_values = extra_col.nunique(dropna=False)
all_zero = bool(extra_col.eq(0).all())
```

All 200,000 values in the `extra` column were zero, so the column was identified as unnecessary.

3.7 Data Quality Assessment Results

Table 2: Summary of Data Quality Assessment Results

Quality Check	Result
Total records	200,000
Missing values	0
Fully duplicated records	707
Repeated user-game playtime pairs	12
Invalid behavior records	0
Invalid numerical values	0
Invalid purchase values	0
Negative playtime records	0
Zero playtime records	0
Playtime without purchase	0
Unique values in extra column	1

The assessment identified three issues requiring preprocessing: 707 duplicate purchase records, 12 repeated playtime pairs, and an unnecessary column.

These findings guided the cleaning operations in the next chapter.

4 Data Cleaning

Based on the data quality assessment, I removed an unnecessary column, 707 duplicate purchase records, and 12 redundant playtime records. The cleaning process was implemented in `datacleaner.py` using Pandas.

4.1 Removing the Unnecessary Column

The `extra` column contained only zero values, so I removed it using `drop()`.

```
def remove_extra_column(df: pd.DataFrame) -> pd.DataFrame:
    return df.drop(columns=["extra"]).copy()
```

This reduced the dataset from five columns to four without removing any records.

4.2 Removing Fully Duplicated Records

I used `drop_duplicates()` to remove 707 identical purchase records, keeping the first occurrence of each record.

```
def remove_duplicate_records(df: pd.DataFrame) -> pd.DataFrame:
    return df.drop_duplicates().copy()
```

This reduced the dataset to 199,293 rows.

4.3 Handling Repeated Playtime Records

I identified 24 play records belonging to 12 repeated user-game pairs with different playtime values.

Since summing these values could double-count overlapping measurements, I decided to retain the maximum recorded playtime for each pair.

The implementation is shown below.

```
def keep_max_playtime(df: pd.DataFrame) -> pd.DataFrame:
    #Select play records
    play_df = df[df["behavior"] == "play"]
    #Find highest playtime index for each user-game pair
    max_indices = play_df.groupby(
        ["user_id", "game_name"]
    )["value"].idxmax()
    #Find redundant play record indices
    indices_to_drop = play_df.index.difference(max_indices)
    #Remove redundant play records
    return df.drop(index=indices_to_drop).copy()
```

The `groupby()` and `idxmax()` methods identify the highest playtime per user-game pair, while `drop()` removes the remaining repeated play records.

This removed 12 records without affecting purchase records. Retaining the maximum is a preprocessing assumption because the original measurement method is unknown.

4.4 Cleaning Results

Table 3: Dataset Size After Cleaning Operations

Processing Stage	Rows	Columns
Original dataset	200,000	5
Remove extra column	200,000	4
Remove full duplicates	199,293	4
Remove repeated play records	199,281	4

In total, 719 redundant records were removed. The cleaned dataset contains 199,281 rows and four columns, preserving all 12,393 users.

I saved the result as `data/processed/clean_steamdata.csv`, keeping the original dataset unchanged.

5 Feature Engineering

The cleaned dataset contained 199,281 user-game interactions. I transformed these records into a user-level dataset to analyze individual Steam users rather than separate interactions.

5.1 User-Level Feature Creation

I used Pandas `groupby()` to group records by `user_id`, separating purchase and play interactions for each user.

The main operations from `create_user_features()` are shown below.

```
def create_user_features(df: pd.DataFrame) -> pd.DataFrame:
    user_features = {}

    for user_id, user_records in df.groupby("user_id"):
        # ... separate purchase and play records

        games_purchased = purchase_records["game_name"].nunique()
        games_played = play_records["game_name"].nunique()
        total_playtime = play_records["value"].sum()
        avg_playtime = (
            play_records["value"].mean()
            if games_played > 0 else 0.0
        )
        max_playtime = (
            play_records["value"].max()
            if games_played > 0 else 0.0
        )
        play_ratio = (
            games_played / games_purchased
            if games_purchased > 0 else 0.0
        )
        # ... store the six features in user_features[user_id]
```

```
    user_df = pd.DataFrame.from_dict(
        user_features, orient="index"
    )
    user_df.index.name = "user_id"
    user_df = user_df.reset_index()
    return user_df
```

The function uses `nunique()` to count distinct games and `sum()`, `mean()`, and `max()` to calculate playtime statistics. The results are stored in a dictionary and converted into a DataFrame with one row per user.

5.2 Engineered Behavioral Features

Six numerical features were created to describe game ownership, recorded playtime, and engagement.

Table 4: Engineered User-Level Features

Feature	Description
`games_purchased`	Number of distinct games owned by the user.
`games_played`	Number of distinct games with recorded playtime.
`total_playtime`	Sum of recorded playtime across played games, measured in hours.
`avg_playtime`	Mean recorded playtime per played game, measured in hours.
`max_playtime`	Highest recorded playtime for a single game, measured in hours.
`play_ratio`	Proportion of purchased games with recorded playtime.

The play ratio was calculated as games played/games purchased, measuring the proportion of owned games with recorded playtime.

For users without play records, average and maximum playtime were set to 0.0 to avoid missing values and retain these observations.

5.3 Feature Engineering Validation

I implemented `validate_user_features()` to check the consistency of the new dataset.

All validation checks passed, confirming that no duplicate users, missing values, invalid play ratios, or inconsistent playtime values were found.

5.4 Feature Engineering Results

The transformation produced 12,393 user-level observations with six behavioral features and one user identifier.

Table 5: Feature Engineering Results

Metric	Result
Cleaned interaction records	199,281
Original unique users	12,393
User-level observations	12,393
Behavioral features	6
Total output columns	7
Duplicate user IDs	0
Missing values	0
Total games purchased	128,804
Total games played	70,477

All 12,393 users were preserved, and the purchased and played game totals matched the earlier data quality assessment.

The final dataset was exported as `data/processed/user_features.csv` for exploratory data analysis.

6 Exploratory Data Analysis

EDA was performed on 12,393 Steam users to examine their gaming behavior, feature distributions, relationships, and potential outliers. The `user_id` column was excluded from statistical analysis.

6.1 Descriptive Statistics

I used Pandas `describe()` to calculate descriptive statistics for the six features.

```
def get_feature_statistics(df: pd.DataFrame) -> pd.DataFrame:
    feature_df = df.drop(columns=["user_id"])
    return feature_df.describe().T
```

Table 6: Descriptive Statistics of User-Level Features

Feature	Mean	Std	Q1	Median	Q3	Max
games_purchased	10.39	35.64	1	2	6	1068
games_played	5.69	17.08	1	1	3	498
total_playtime	278.01	760.27	1.3	13.5	155.6	11754
avg_playtime	74.32	283.75	1.1	7.9	39	11754
max_playtime	176.16	504.76	1.2	11.6	107	11754
play_ratio	0.727	0.345	0.5	1.0	1.0	1.0

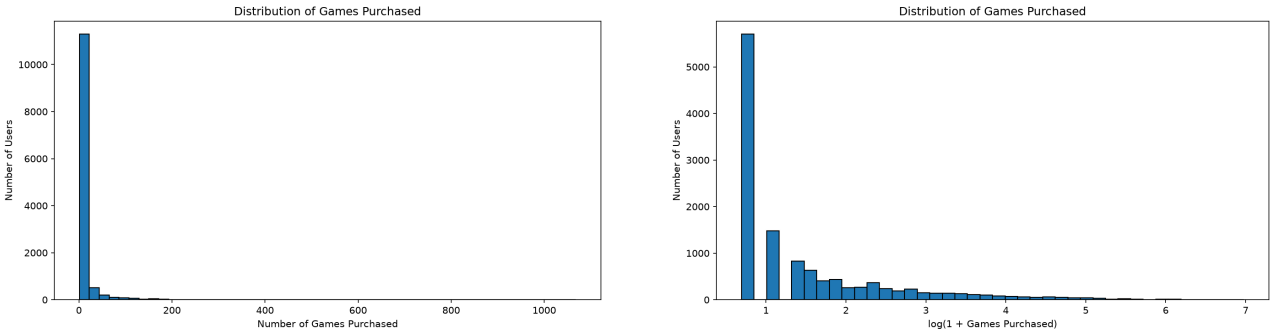
The mean number of purchased games was 10.39, compared to a median of 2. Similarly, mean playtime was 278.01 hours, compared to a median of 13.5 hours. These differences indicate strongly right-skewed distributions.

6.2 Distribution of Games Purchased

I initially visualized game ownership using a standard histogram. However, the maximum of 1,068 games compressed most observations into the first bins.

A logarithmic histogram improved visibility but made actual game counts harder to interpret.

Figure 2 compares these two initial approaches.



(a) Original distribution

(b) Log-transformed distribution

Figure 2: Original and log-transformed distributions of purchased games.

Therefore, I used categorical grouping with `pd.cut()` to produce a more interpretable visualization.

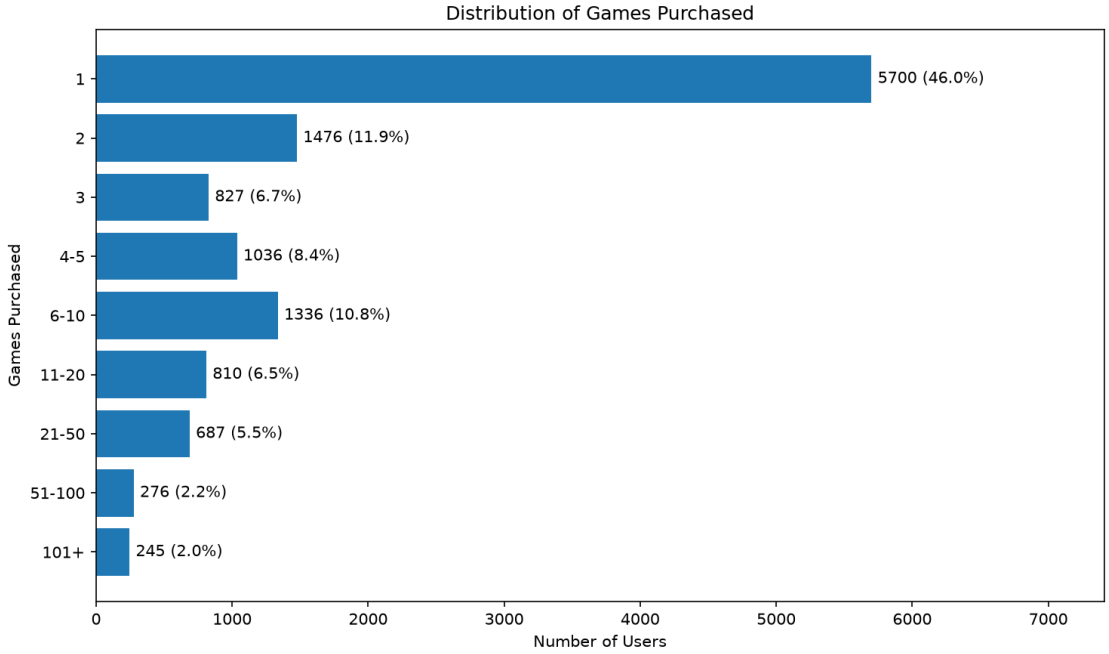


Figure 3: Distribution of users by purchased game categories.

The categorical chart shows that 5,700 users (46.0%) owned exactly one game, while 90.3% owned no more than 20 games. Only 245 users (2.0%) owned over 100 games, confirming the highly uneven distribution.

6.3 Distribution of Total Playtime

I applied the same categorical approach to total recorded playtime.

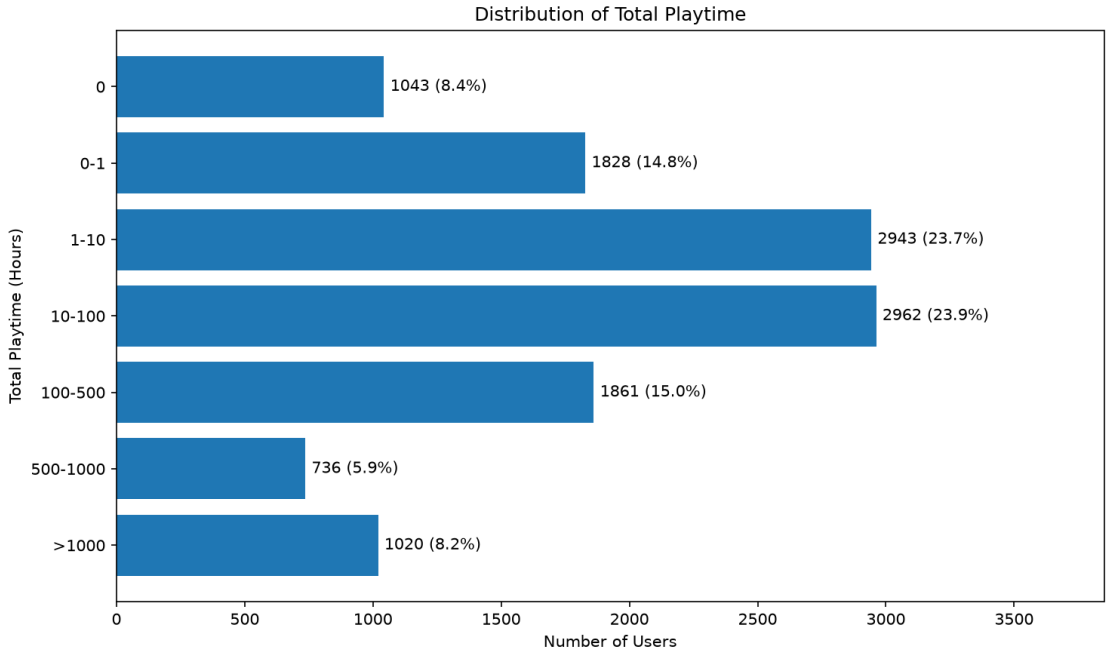


Figure 4: Distribution of users by total recorded playtime in hours.

The results show that 1,043 users (8.4%) had zero recorded playtime, while 70.8% had no more than 100 hours. However, 1,020 users (8.2%) exceeded 1,000 hours, demonstrating substantial differences in recorded gaming activity.

Zero recorded playtime does not necessarily mean that a user has never played.

6.4 Relationship Between Ownership and Playtime

I used `pd.cut()`, `groupby()`, and `agg()` to compare total playtime across nine game ownership categories.

Table 7: Total Playtime by Game Ownership Category

Games Owned	Users	Mean (h)	Median (h)
1	5700	66.17	2.70
2	1476	139.73	9.40
3	827	163.18	20.20
4–5	1036	179.45	28.85
6–10	1336	325.86	58.25
11–20	810	572.91	240.20
21–50	687	959.76	595.70
51–100	276	1490.38	1194.95
101+	245	2330.63	1860.50

Median playtime increased from 2.70 hours for users owning one game to 1,860.50 hours for users owning over 100 games. This suggests a positive association between game ownership and recorded playtime, but does not establish causation.

6.5 Correlation Analysis

I used Spearman correlation to examine monotonic relationships between features, considering the skewed distributions and extreme values.

```
def get_feature_correlations(df: pd.DataFrame) -> pd.DataFrame:
    feature_df = df.drop(columns=["user_id"])
    correlation_matrix = feature_df.corr(method="spearman")
    return correlation_matrix
```

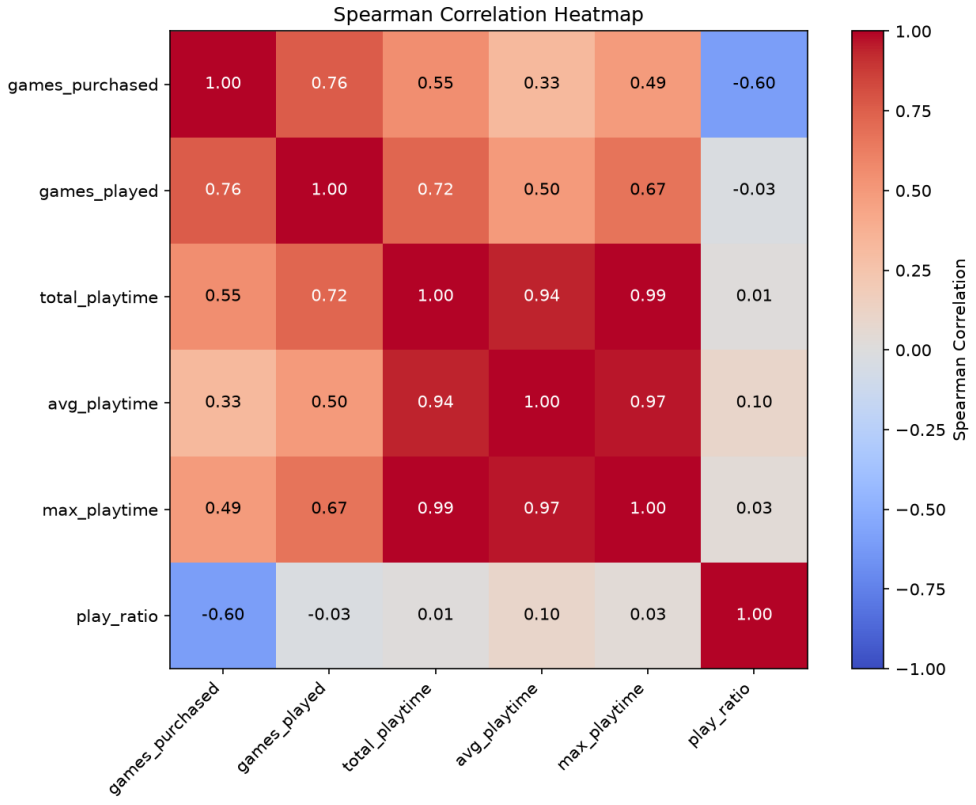


Figure 5: Spearman correlation heatmap of the six behavioral features.

Very high correlations were observed between `total_playtime` and `max_playtime` (0.99), and between `avg_playtime` and `max_playtime` (0.97).

Additionally, `games_purchased` and `games_played` had a correlation of 0.76, while `games_purchased` and `play_ratio` showed -0.60.

These results indicate potentially redundant features that should be considered during future model development.

6.6 Outlier Analysis

I applied the Interquartile Range (IQR) method to identify potential outliers in game ownership and total playtime.

The upper boundary was calculated as $Q_3 + 1.5(Q_3 - Q_1)$.

The key operations from `analyze_outliers()` were:

```
# Inside analyze_outliers(), for each feature
q1 = df[feature].quantile(0.25)
q3 = df[feature].quantile(0.75)

iqr = q3 - q1
lower_bound = q1 - 1.5 * iqr
upper_bound = q3 + 1.5 * iqr

outliers = df[
    (df[feature] < lower_bound) |
    (df[feature] > upper_bound)
]
# ... calculate counts and percentages
```

Table 8: IQR-Based Outlier Analysis Results

Feature	Upper Bound	Outliers	Percentage
games_purchased	13.50	1672	13.49%
total_playtime	387.05	2072	16.72%

The analysis identified 1,672 users (13.49%) with at least 14 purchased games and 2,072 users (16.72%) with over 387.05 hours of recorded playtime.

These observations were retained because they may represent legitimate, highly active players rather than data errors. Their influence will be investigated during modeling.

7 Conclusions and Future Modeling Plan

7.1 Summary of Findings

After cleaning and feature engineering, the dataset contains 12,393 users and six behavioral features. The EDA revealed strongly skewed distributions, extreme values, and high correlations between playtime features, which must be considered during modeling.

7.2 Future Modeling Plan

I will use K-Means clustering to investigate whether Steam users can be grouped based on their gaming behavior. Since this is an unsupervised learning problem, no predefined target labels are available.

I will first build a baseline K-Means model using all six numerical features, excluding `user_id`. I will apply StandardScaler and initially use three clusters ($K = 3$) but that does not mean that there are only 3 groups.

I will evaluate the model using the Silhouette Score and examine the size and behavioral characteristics of each cluster.

To improve the model, I will test logarithmic transformations, alternative scaling methods, and reduced feature sets to address skewed distributions, extreme values, and highly correlated features.

I will also test different numbers of clusters ($K = 2$ to $K = 8$) and compare the results with the baseline using Silhouette Scores and cluster interpretability.

Finally, I will examine whether the resulting clusters represent meaningful differences in Steam player behavior and discuss the limitations of the analysis.